AGENT EVOLUTION · NOTEBOOK 4 OF 8

Version 4 — Validate SQL Before Running It

A guide to `agent_pinecone_validate_query_with_workflow_latest.ipynb`: how one SQL policy and PostgreSQL `EXPLAIN` check every query before it runs, why `run_query` checks again, and what validation still cannot catch.

PA

Tutor: Prudhvi Akella
Lead Data & AI Engineer · 13+ years in data engineering and AI architecture

Stack: LangChain `create_agent` · Bedrock chat model · OpenAI embeddings · Pinecone · PostgreSQL

Tools: `lookup_schema` · `lookup_examples` · `lookup_common` · `validate_query` · `run_query`

1. What changes from Version 3
2. The end-to-end workflow
3. Two checks, two costs
4. The SQL policy, step by step
5. The scan copy
6. What the policy blocks
7. EXPLAIN: parse and plan, never run
8. What `validate_query` returns
9. Why `run_query` checks again
10. The loop with validation
11. One call, step by step
12. What changed in the prompt
13. What validation cannot catch
14. The limitation, and what Version 5 changes
15. Key takeaways

1 What changes from Version 3

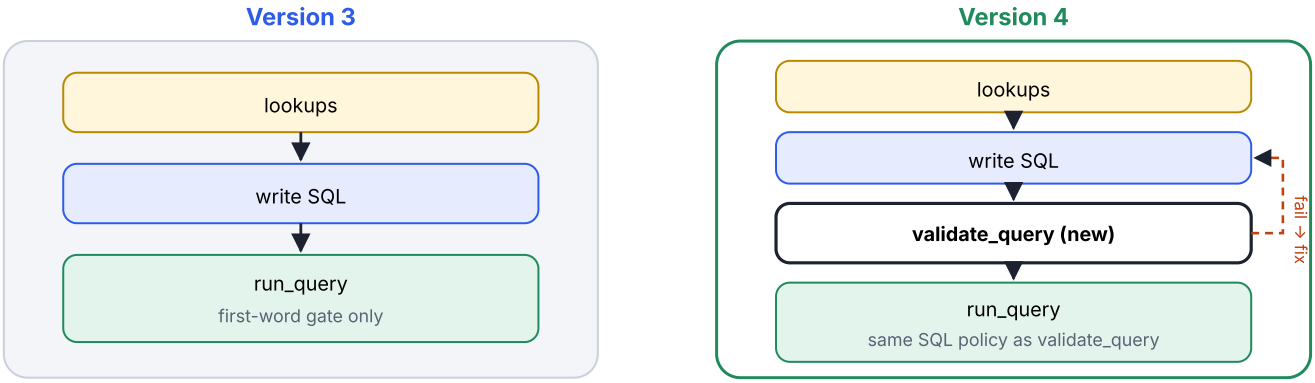


Figure 1. One new tool sits between writing SQL and running it.

	Version 3	Version 4
Check before running	first word only	`validate_query`: SQL policy + PostgreSQL `EXPLAIN`
Multiple statements	not detected	rejected
Writes hidden in a CTE	not detected	rejected

Unknown column / type error	found when the query runs	found by `EXPLAIN`, before running
`run_query` safety check	first word	the same policy as `validate_query`
Prompt	—	"call `validate_query` before `run_query`"
Tools	4	5

2 The end-to-end workflow

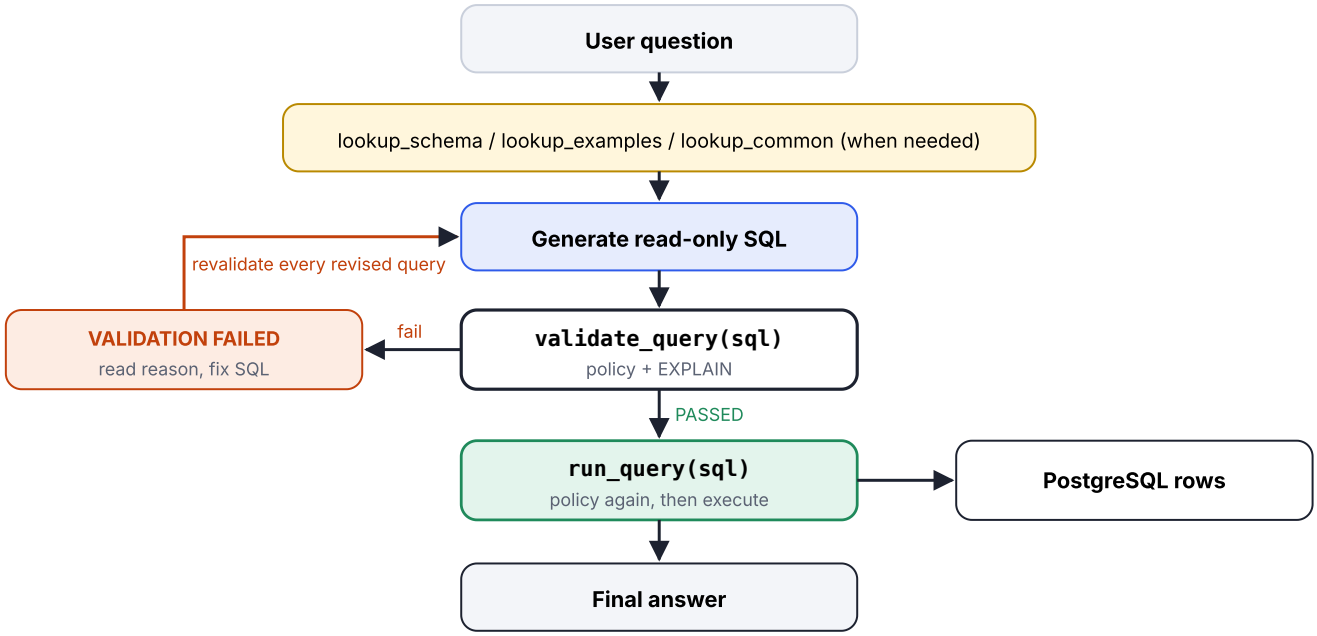


Figure 2. Version 4 end to end. Every query passes `validate_query` before `run_query` runs it.

1. **Look up what is needed**, exactly as in Version 3.
2. **Write one read-only query.**
3. **Validate it.** `validate_query` checks the SQL policy, then asks PostgreSQL to `EXPLAIN` it.
4. **If validation fails,** the model reads the reason, fixes the SQL and validates again.
5. **If it passes,** the model calls `run_query` with the same SQL, then answers from the rows.

A bad query now usually fails at step 3, before it ever runs on the data.

3 Two checks, two costs

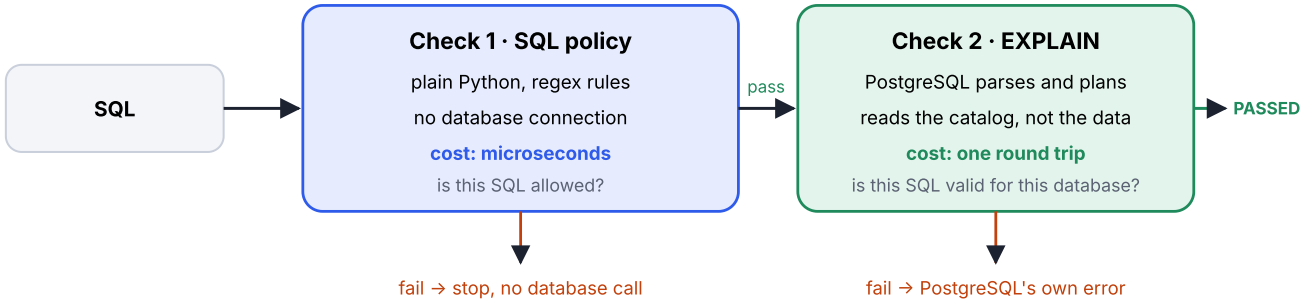


Figure 3. The cheap check runs first. Only SQL that passes it reaches PostgreSQL.

The two checks answer different questions:

- **The policy** asks “is this kind of SQL allowed at all?” It looks at the text only. It needs no connection, so it is instant.
- `EXPLAIN` asks “does this SQL make sense for *this* database?” Only PostgreSQL knows the tables, columns and types, so this needs a round trip.

Running the policy first means unsafe SQL never reaches the database, not even to be explained.

4 The SQL policy, step by step

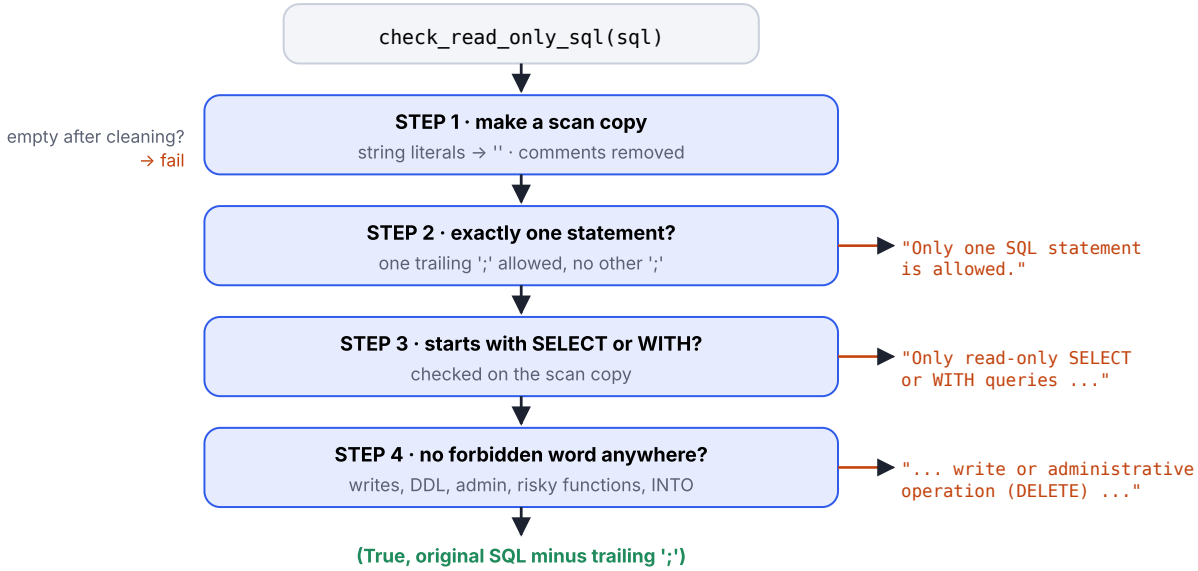


Figure 4. Four steps. Each failure returns a specific reason the model can act on.

These are the real results of `check_read_only_sql` from the notebook's code:

SQL	OK?	Reason returned
`SELECT COUNT(*) FROM orders`	✓	—
`SELECT * FROM orders WHERE note = 'drop; delete'`	✓	— (the words are inside a string)
`SELECT last_updated FROM orders`	✓	— (`update` inside a name is not a match)
`SELECT 1; SELECT 2`	×	Only one SQL statement is allowed.
`DELETE FROM orders`	×	Only read-only SELECT or WITH queries are allowed.
`WITH x AS (DELETE FROM orders RETURNING *) SELECT * FROM x`	×	... write or administrative operation (DELETE) ...
`SELECT * INTO bk FROM orders`	×	... write or administrative operation (INTO) ...
`SELECT pg_sleep(30)`	×	... write or administrative operation (PG_SLEEP) ...

When the policy passes, it returns the **original** SQL (string values intact), minus a trailing `;`. That is the text that goes to `EXPLAIN` and later runs. The scan copy is used only for checking.

> The `pg_sleep` message says “write or administrative operation”, which is not quite accurate: `pg_sleep` just waits. It is blocked because it can tie up a connection. The model still gets a clear “rejected” and the word that caused it.

5 The scan copy

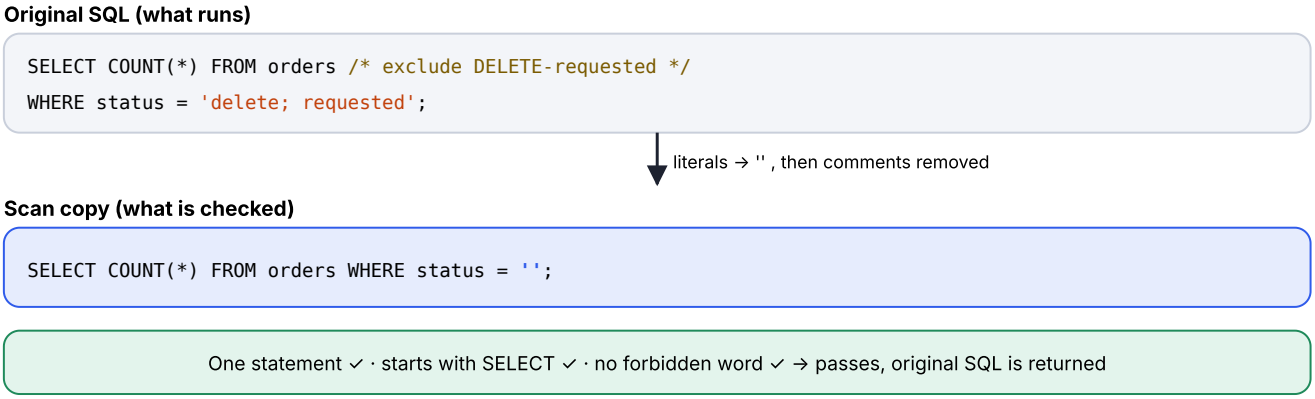


Figure 5. The words *DELETE* and *;* appear only in a comment and a string value, so they are ignored.

A naive check would reject this query twice: once for the word `DELETE` and once for the `;` inside the string. Both are false alarms. The scan copy removes them before any rule runs:

1. **String literals become `''`.** Anything between single quotes is a value, not SQL. Doubled quotes inside a string (`'it''s'`) are handled.
2. **Comments are removed**: both `/* … */` and `-- …` to the end of the line.

Literals are replaced **before** comments are removed. Otherwise a value like `'a--b'` would look like the start of a comment and cut the rest of the line.

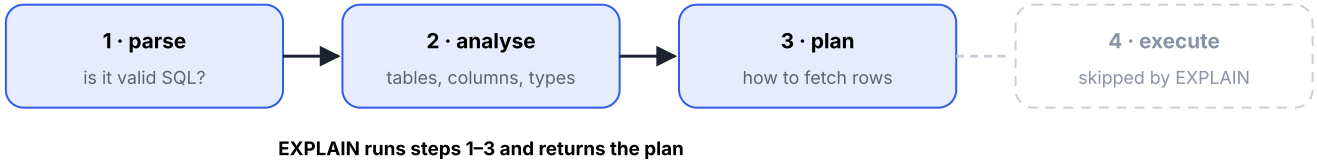
> Word boundaries matter too. The forbidden list uses `\b…\b` , so `UPDATE` matches the word `update` but not `last_updated` or `updated_at` . Without this, ordinary column names would be blocked.

6 What the policy blocks

Group	Words	Why
Data writes	`INSERT UPDATE DELETE MERGE`	change rows; can hide inside a `WITH`
Structure (DDL)	`CREATE ALTER DROP TRUNCATE`	change or destroy tables
Permissions	`GRANT REVOKE`	change who can do what
Admin	`CALL COPY VACUUM REFRESH LOCK`	run procedures, read/write files, maintenance, block other users
Hidden table creation	`INTO`	`SELECT … INTO new_table` creates a table
Risky functions	`pg_sleep pg_read_file pg_read_binary_file pg_ls_dir lo_import lo_export dblink* set_config pg_terminate_backend pg_cancel_backend`	tie up connections, read server files, reach other databases, change settings, kill sessions

> **What is deliberately not on the list:** `SET` , `DO` and `ANALYZE` . They were in the original draft but matched normal SQL and column names, and they cannot start a `SELECT` / `WITH` query anyway (step 3 already blocks them). The policy is tuned to stop real risks without rejecting good queries.

7 EXPLAIN: parse and plan, never run



EXPLAIN runs steps 1–3 and returns the plan

```
EXPLAIN SELECT status, COUNT(*) FROM orders GROUP BY status
HashAggregate (cost=48.25..50.25 rows=200 width=40)
Group Key: status
-> Seq Scan on orders (cost=0.00..37.50 rows=2150 width=32)
```

Figure 6. `EXPLAIN` stops before execution. No business row is read. Plan numbers are illustrative.

PostgreSQL handles every query in stages. `EXPLAIN` runs the first three and returns the plan, without running the query. That gives a full validity check from the database itself, without touching any business data.

EXPLAIN catches	Example error
Unknown column	`column "order_status" does not exist`
Unknown table	`relation "order" does not exist`
Syntax errors	`syntax error at or near "FORM"`
Type errors	`operator does not exist: integer = text`
Grouping errors	`column "p.name" must appear in the GROUP BY clause …`
Ambiguous names in joins	`column reference "id" is ambiguous`

Plain `EXPLAIN` only. `EXPLAIN ANALYZE` *does* run the query to measure it. The tool always sends `"EXPLAIN " + sql`, and the policy has already rejected anything that does not start with `SELECT` or `WITH`, so `ANALYZE` cannot be slipped in.

`validate_query` connects with the same read-only settings as `run_query`, with a shorter 10-second timeout.

8 What `validate_query` returns

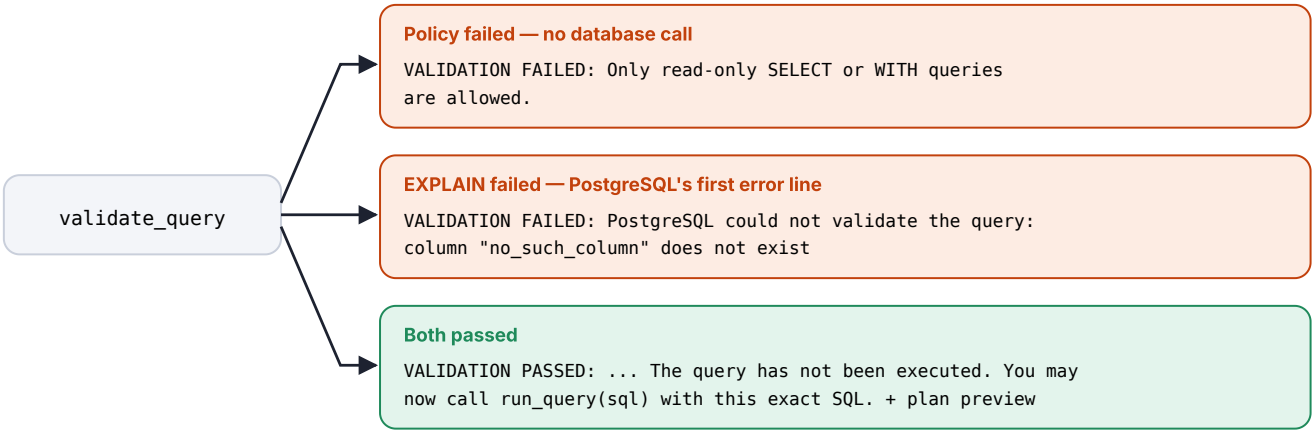


Figure 7. The three real outputs of `validate_query`, taken from the notebook's code.

Every result starts with `VALIDATION FAILED` or `VALIDATION PASSED`. That fixed prefix makes the outcome easy for the model to read, and easy for code to count later (Version 8 counts the failures as `repair_count`).

- **Failure messages are short and specific.** Only the first line of the PostgreSQL error is kept, cut at 500 characters. Connection details are never included.
- **The pass message says what to do next:** call `run_query` with this exact SQL. It also says plainly that the query has *not* run, so the model does not mistake a plan for a result.
- **The plan preview** is up to 8 plan lines (max 3,000 characters). It lets the model notice something like a full scan of a large table before running it.

> An earlier draft of this tool told the model to call `execute_read_only_sql`, a tool that does not exist. A message that names the wrong next step sends the model to a dead end. Tool output is part of the instructions; it must use the real tool names.

9 Why `run_query` checks again

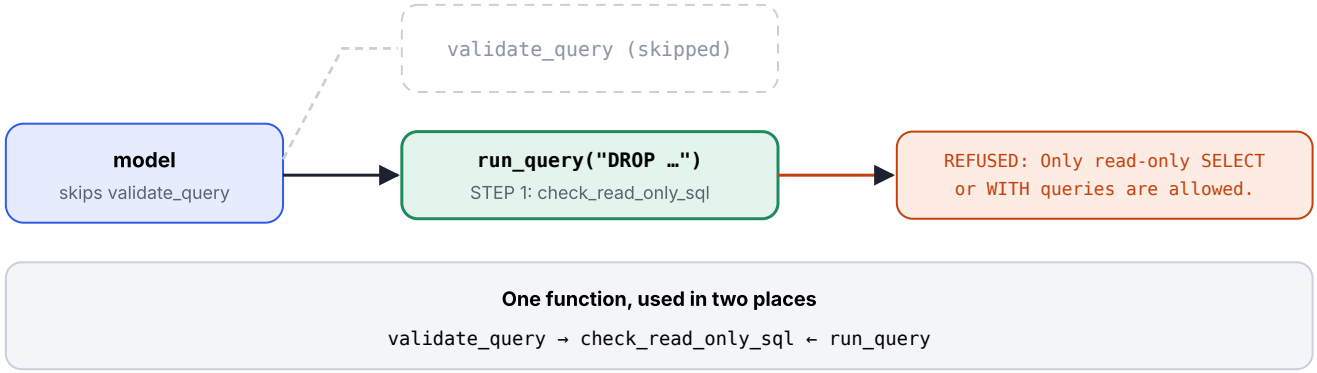


Figure 8. The prompt asks for validation first; `run_query` enforces the policy anyway.

The prompt says "call `validate_query` before `run_query`". But a prompt is a request. The model can skip a step, especially after several retries. If `run_query` trusted that validation had happened, one skipped step would send unchecked SQL to the database.

So `run_query` calls the same `check_read_only_sql` function as its first step. Because both tools share **one** function, they can never disagree about what is allowed. Change the policy once, and both tools follow.

	`validate_query`	`run_query`
Runs the SQL policy	yes	yes (same function)
Runs `EXPLAIN`	yes	no — execution reports the same errors anyway
Executes the query	no	yes
Failure prefix	`VALIDATION FAILED:`	`REFUSED:` or `SQL error:`

The order itself is still only a prompt rule. Nothing in the code checks that the SQL sent to `run_query` was validated first, or that it is the same SQL that passed. The policy blocks unsafe SQL either way, and execution reports any remaining errors. Enforcing "validated first" in code is a job for middleware, which arrives in Version 6.

10 The loop with validation

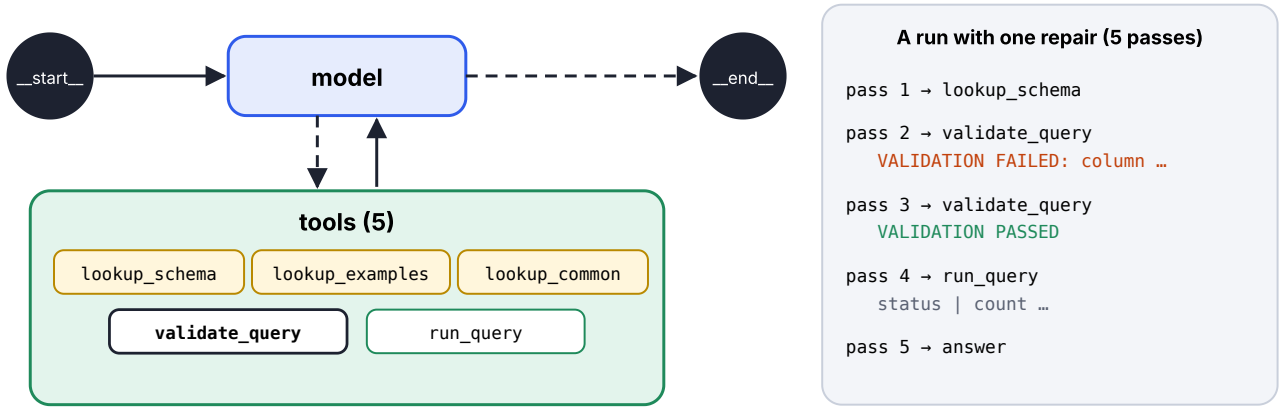


Figure 9. Still four graph nodes; five tools inside. The pass list is the message order from a test run of the notebook's agent.

Adding `validate_query` adds one pass to every query: a clean run is now lookup → validate → run → answer. Each failed validation adds one more pass, but costs only a plan, not a query on the data.

Run	Passes	vs Version 3
lookup → validate (pass) → run → answer	4	+1 pass for validation
lookup → validate (fail) → validate (pass) → run → answer	5	the failed attempt never touched the data
validate (fail: policy) → validate (pass) → run → answer	4	the policy failure needed no database call
run fails after validation passed → revalidate → run → answer	6+	rare: e.g. a runtime error like division by zero

11 One call, step by step

```
result["messages"] == [
  HumanMessage("How many orders were cancelled?"),

  AIMessage(tool_calls=[lookup_schema(term="order status")]),
  ToolMessage("RESULT 1 | id=col-orders-status | score=0.83 ...", name="lookup_schema"),

  AIMessage(tool_calls=[validate_query(sql="SELECT COUNT(*) FROM orders WHERE order_status =
'cancelled'")]),
  ToolMessage("VALIDATION FAILED: PostgreSQL could not validate the query:
              column \"order_status\" does not exist", name="validate_query"),

  AIMessage(tool_calls=[validate_query(sql="SELECT COUNT(*) FROM orders WHERE status =
'cancelled'")]),
  ToolMessage("VALIDATION PASSED: ... You may now call run_query(sql) with this exact SQL.
              Query plan preview: Aggregate ... Seq Scan on orders ...",
name="validate_query"),

  AIMessage(tool_calls=[run_query(sql="SELECT COUNT(*) FROM orders WHERE status =
'cancelled'")]),
  ToolMessage("count\n7\n(1 rows.)", name="run_query"),

  AIMessage("ANSWER: 7 cancelled orders. ...")
]
```

Figure 10. The message list for a run with one repair, shortened. SQL and values are illustrative.

Three things to check in a trace like this:

- **Every `run_query` follows a `VALIDATION PASSED`** for the same SQL.
- **Every `VALIDATION FAILED` is followed by a changed query**, not the same SQL again.
- **The SQL in the answer's SOURCE** is the SQL from the last successful `run_query`.

12 What changed in the prompt

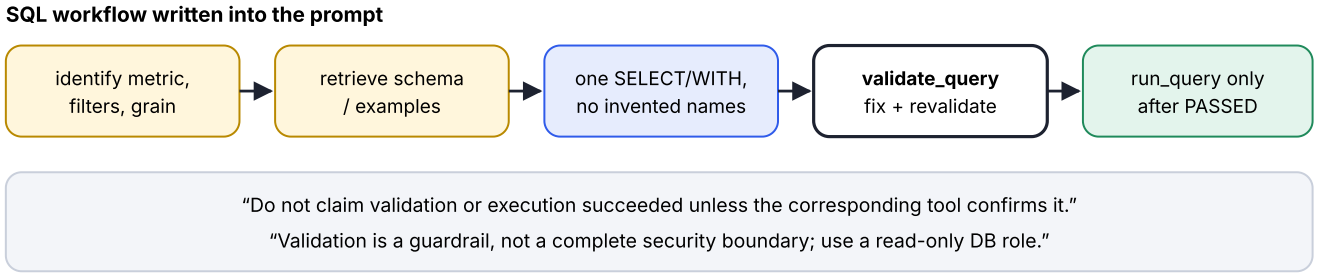


Figure 11. The prompt turns the workflow into an ordered procedure.

Block	What changed from Version 3
AVAILABLE TOOLS	Adds `validate_query(sql)` : "applies conservative single-statement read-only checks and asks PostgreSQL EXPLAIN to parse/plan the query. It does not run the business query."
TOOL-SELECTION AND SQL WORKFLOW	Adds the order: validate before run; on failure, correct using the error and validate again; run only after success; revalidate every revised query.
Honesty rule	Now covers validation too: never claim validation *or* execution succeeded unless the tool confirms it.
Safety note	Validation is a guardrail, not a security boundary; the read-only database role still is.

> The prompt, the `validate_query` docstring ("Call this before run_query") and the pass message ("You may now call run_query(sql)...") all say the same thing in three places. Consistent wording across prompt, docstring and tool output is what keeps the model on the intended path.

13 What validation cannot catch

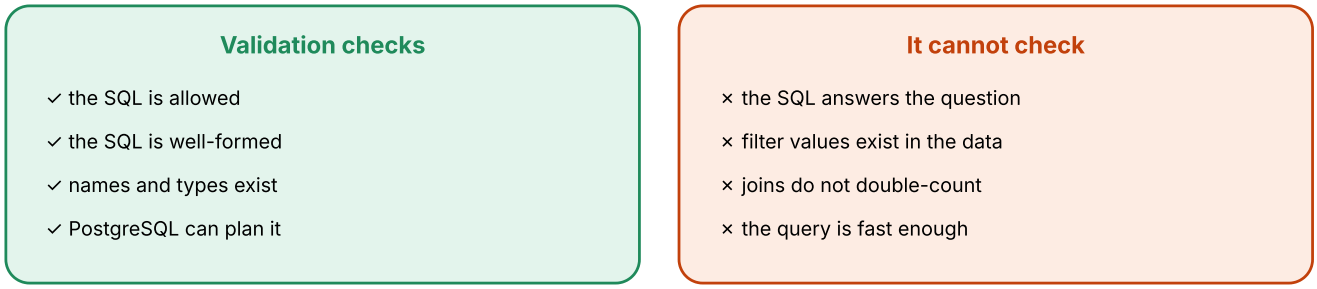


Figure 12. Valid SQL is not the same as correct SQL.

Valid SQL that is still wrong	Why validation passes it
`WHERE status = 'canceled'` when the data uses `'cancelled'`	a string value is never checked against the data; the query returns 0 rows with no error
`COUNT(*)` after joining `orders` to `order_items`	the join is legal; it counts lines instead of orders
`WHERE order_date >= '2025-01-01'` copied from an example	a legal filter the user never asked for
`SUM(line_total)` when "units" was asked	the column exists; it measures the wrong thing
a full scan of a very large table	`EXPLAIN` shows the cost but the tool sets no limit

These are **meaning** errors, not **syntax** errors. The defences against them are the ones from earlier versions: the prompt rules on grain, joins and denominators (Version 1), retrieved schema and definitions (Version 3), and checking the answer against the trace (Version 2).

> **A green "VALIDATION PASSED" is not a correct answer.** It only means the database accepted the query. Always read the SQL and the rows.

14 The limitation, and what Version 5 changes

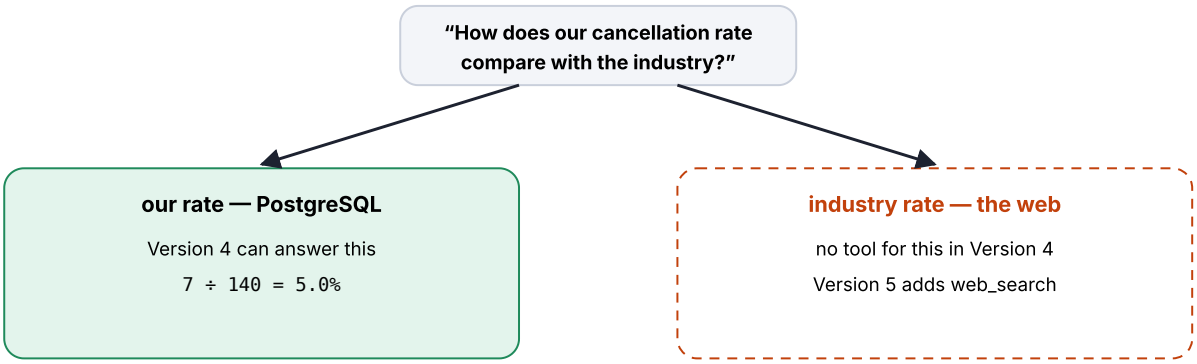


Figure 13. Some questions need facts that are not in the database. Values are illustrative.

Version 4 answers internal questions safely: real names from the knowledge base, checked SQL, real rows. But every fact it can reach lives inside the company's database or knowledge base. It has no way to look up:

- industry benchmarks ("typical cancellation rate in online retail");
- competitor or market information;
- anything current or public.

	Version 4	Version 5
Evidence sources	PostgreSQL + knowledge base	PostgreSQL + knowledge base + the public web
New tool	—	`web_search(query, include_domains, exclude_domains, max_results)`
New prompt skill	—	**routing**: internal data, the web, or both
SOURCE section	SQL only	SQL and web URLs, labelled separately

15 Key takeaways

1. **Check before you run.** `validate_query` sits between writing SQL and running it.
2. **Cheap check first.** The SQL policy needs no database; `EXPLAIN` runs only on SQL the policy allows.
3. **The policy has four steps:** scan copy, one statement, `SELECT` / `WITH` only, no forbidden word anywhere.
4. **Check a scan copy, run the original.** String values and comments are removed for checking only, so `'delete requested'` is not a false alarm.
5. **`EXPLAIN` parses and plans but never executes.** It catches unknown names, type errors and syntax errors without reading business rows.
6. **Fixed prefixes** (`VALIDATION FAILED` / `PASSED`) make results easy for the model to read and for code to count.
7. **Tool output must name the real next tool.** A message pointing to a tool that does not exist is a dead end.
8. **One policy, two tools.** `run_query` re-checks with the same function, so skipping validation cannot bypass it.
9. **The order "validate, then run" is still only a prompt rule.** Enforcing order in code is a middleware job (Version 6).
10. **Valid is not correct.** Wrong values, double-counting joins and wrong measures all pass validation.

> **Next — Version 5:** `web_search` brings in public web sources, and the agent learns to route each question to internal data, the web, or both.